

MODULE 5

Artificial Neural Networks

Artificial Neural Networks (ANNs) imitate human brain behaviour and the way in which learning happens in a human. The human brain constitutes a mass of neurons that are all connected as a network, which is actually a directed graph. These neurons are the processing units which receive information, process it and then transmit this data to other neurons that allows humans to learn almost any task. ANN is a learning mechanism that models a human brain to solve any non-linear and complex problem. Each neuron is modelled as a computing unit, or simply called as a node in ANN, that is capable of doing complex calculations. ANN is a system that consists of many such computing units operating in parallel that can learn from observations. They are widely used in developing artificial learning systems and have inspired researchers and industry in Machine Learning nowadays. Some typical applications of ANN in the field of computer science are Natural Language Processing (NLP), pattern recognition, face recognition, speech recognition, character recognition, text processing, stock prediction, computer vision, etc. ANNs also have been considerably used in other engineering fields such as Chemical industry, Medicine, Robotics, Communications, Banking, and Marketing. This chapter aims to introduce the concepts necessary to understand the working of Artificial Neural Networks.

Learning Objectives

- Understand the basics of human nervous system and biological neurons
- Learn about McCulloch & Pitts Neuron Mathematical Model of an artificial neuron
- Know about the structure of Artificial Neural Network
- Introduce different types of activation functions
- Explore about the first neural network model called 'Perceptron' which is a linear binary classifier used for supervised learning
- Introduce different types of Artificial Neural Networks
- Understand the concepts of learning in Multi-Layer Perceptron (MLP) using back propagation
- Study about Radial Basis Function Neural Network (RBFNN) which is a type of multi-layer perceptron particularly useful for interpolation, function approximation, time series prediction, classification and system control
- Explore Self Organizing Feature Map (SOFM) which is a competitive learning network for unsupervised learning

10.1 INTRODUCTION

The human nervous system has billions of neurons that are the processing units which make humans to perceive things, to hear, to see and to smell. The human nervous system works beautifully, making us understand who we are, what we do, where we are and everything in our surrounding. It makes us to remember, recognize and correlate things around us. It is a learning system that consists of functional units called nerve cells, typically called as neurons. The human nervous system is divided into two sections called the Central Nervous System (CNS) and the Peripheral Nervous System (PNS). The brain and the spinal cord constitute the CNS and the neurons inside and outside the CNS constitute the PNS. The neurons are basically classified into three types called sensory neurons, motor neurons and interneurons. Sensory neurons get information from different parts of the body and bring it into the CNS, whereas motor neurons receive information from other neurons and transmit commands to the body parts. The CNS consists of only interneurons which connect one neuron to another neuron by receiving information from one neuron and transmitting it to another. The basic functionality of a neuron is to receive information, process it and then transmit it to another neuron or to a body part.

Scan for information on 'Convolution Neural Network', 'Modular Neural Network', and 'Recurrent Neural Network'



10.2 BIOLOGICAL NEURONS

A typical biological neuron has four parts called dendrites, soma, axon and synapse. The body of the neuron is called as soma. Dendrites accept the input information and process it in the cell body called soma. A single neuron is connected by axons to around 10,000 neurons and through these axons the processed information is passed from one neuron to another neuron. A neuron gets fired if the input information crosses a threshold value and transmits signals to another neuron through a synapse. A synapse gets fired with an electrical impulse called spikes which are transmitted to another neuron. A single neuron can receive synaptic inputs from one neuron or multiple neurons. These neurons form a network structure which processes input information and gives out a response. The simple structure of a biological neuron is shown in Figure 10.1.

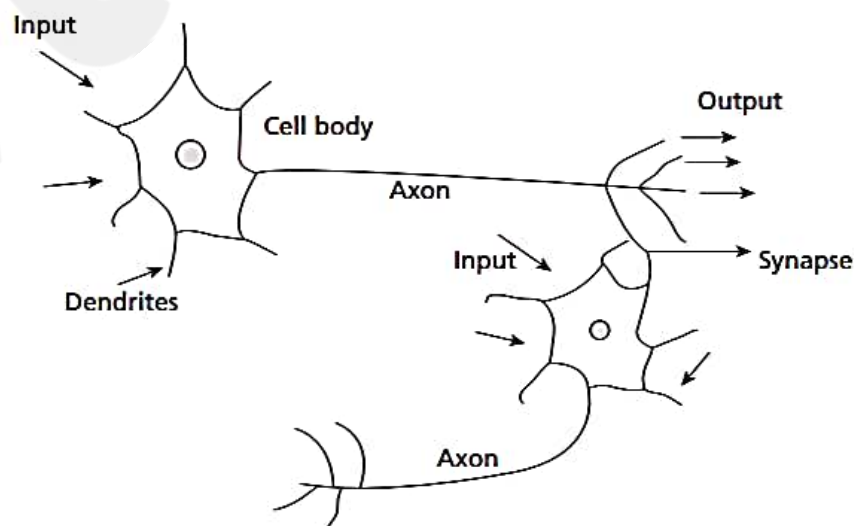


Figure 10.1: A Biological Neuron

10.3 ARTIFICIAL NEURONS

Artificial neurons are like biological neurons which are called as nodes. A node or a neuron can receive one or more input information and process it. Artificial neurons or nodes are connected by connection links to one another. Each connection link is associated with a synaptic weight. The structure of a single neuron is shown in Figure 10.2.

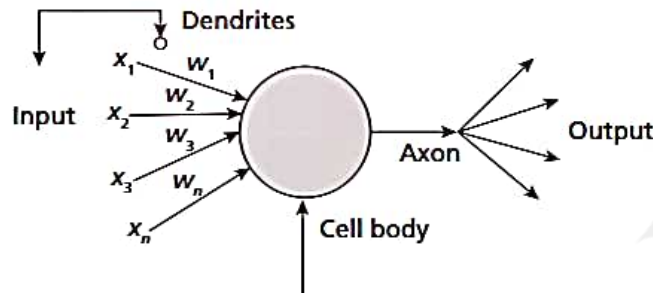


Figure 10.2: An Artificial Neuron

10.3.1 Simple Model of an Artificial Neuron

The first mathematical model of a biological neuron was designed by McCulloch & Pitts in 1943. It includes two steps:

1. It receives weighted inputs from other neurons
2. It operates with a threshold function or activation function

The received inputs are computed as a weighted sum which is given to the activation function and if the sum exceeds the threshold value the neuron gets fired. The mathematical model of a neuron is shown in Figure 10.3.

The neuron is the basic processing unit that receives a set of inputs $x_1, x_2, \dots, x_n$ and their associated weights $w_1, w_2, \dots, w_n$. The Summation function 'Net-sum' Eq. (10.1) computes the weighted sum of the inputs received by the neuron.

$$\text{Net-sum} = \sum_{i=1}^n x_i w_i \quad (10.1)$$

The activation function is a binary step function which outputs a value 1 if the Net-sum is above the threshold value θ , and a 0 if the Net-sum is below the threshold value θ . Therefore, the activation function is applied to Net-sum as shown in Eq. (10.2).

$$f(x) = \text{Activation function (Net - sum)} \quad (10.2)$$

$$\text{Then, output of a neuron } Y = \begin{cases} 1 & \text{if } f(x) \geq \theta \\ 0 & \text{if } f(x) < \theta \end{cases} \quad (10.3)$$

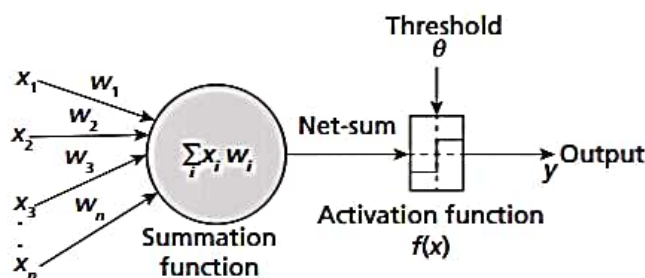


Figure 10.3: McCulloch & Pitts Neuron Mathematical Model

McCulloch & Pitts Neuron model can represent only a few Boolean functions. A Boolean function has binary inputs and provides a binary output. For example, an AND Boolean function neuron would fire when all the inputs are 1, whereas an OR Boolean function neuron would fire even when one input is 1. Moreover, the weight and threshold values are fixed in this mathematical model.

10.3.2 Artificial Neural Network Structure

Artificial Neural Network (ANN) imitates a human brain which inhibits some intelligence. It has a network structure represented as a directed graph with a set of neuron nodes and connection links or edges connecting the nodes as shown in Figure 10.4. The nodes in the graph are arrayed in a layered manner and can process information in parallel. The network given in the figure has three layers called input layer, hidden layer and output layer. The input layer receives the input information ($x_1, x_2, \dots, x_n$) and passes it to the nodes in the hidden layer. The edges connecting the nodes from the input layer to the hidden layer are associated with synaptic weights called as connection weights. These computing nodes or neurons perform some computations based on the input information ($x_1, x_2, \dots, x_n$) received and if the weighted sum of the inputs to a neuron is above the threshold or the activation level of the neuron, then the neuron fires. Each neuron employs an activation function that determines the output of the neuron. The neuron transforms linearly the input signals by computing the sum of the product of input signals and weights and adds biases to it. Then, the activation function maps the weighted input sum to a non-linear output value. The node in the output layer gives the output as a single value.

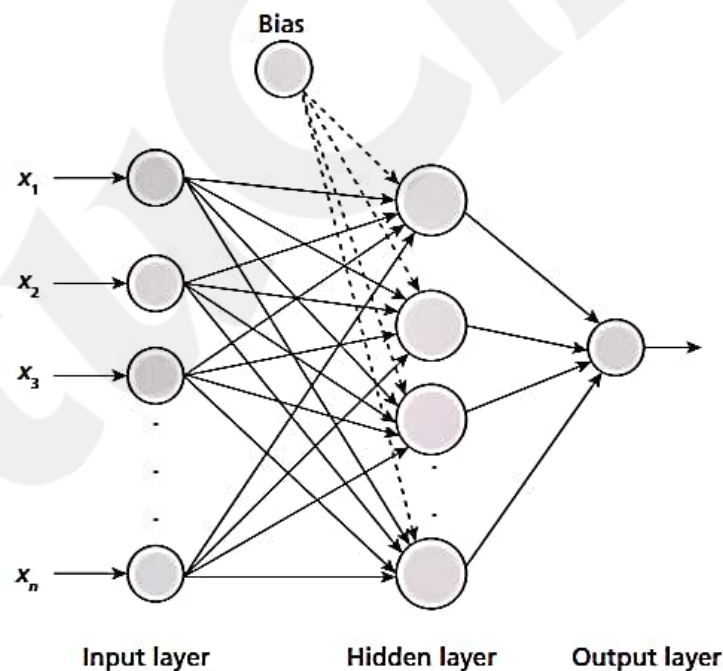


Figure 10.4: Artificial Neural Network Structure

10.3.3 Activation Functions

Activation functions are mathematical functions associated with each neuron in the neural network that map input signals to output signals. It decides whether to fire a neuron or not based on the input signals the neuron receives. These functions normalize the output value of each neuron either between 0 and 1 or between -1 and $+1$. Typical activation functions can be linear or non-linear.

Linear functions are useful when the input values can be classified into any one of the two groups and are generally used in binary perceptrons. Non-linear functions, on the other hand, are continuous functions that map the input in the range of $(0, 1)$ or $(-1, 1)$, etc. These functions are useful in learning high-dimensional data or complex data such as audio, video and images.

Below are some of the activation functions used in ANNs:

1. Identity Function or Linear Function

$$f(x) = x \quad \forall x \quad (10.4)$$

The value of $f(x)$ increases linearly or proportionally with the value of x . This function is useful when we do not want to apply any threshold. The output would be just the weighted sum of input values. The output value ranges between $-\infty$ and $+\infty$.

2. Binary Step Function

$$f(x) = \begin{cases} 1 & \text{if } f(x) \geq \theta \\ 0 & \text{if } f(x) < \theta \end{cases} \quad (10.5)$$

The output value is binary, i.e., 0 or 1 based on the threshold value θ . If value of $f(x)$ is greater than or equal to θ , it outputs 1 or else it outputs 0.

3. Bipolar Step Function

$$f(x) = \begin{cases} 1 & \text{if } f(x) \geq \theta \\ -1 & \text{if } f(x) < \theta \end{cases} \quad (10.6)$$

The output value is bipolar, i.e., +1 or -1 based on the threshold value θ . If value of $f(x)$ is greater than or equal to θ , it outputs +1 or else it outputs -1.

4. Sigmoidal Function or Logistic Function

$$\sigma(x) = \frac{1}{1 + e^{-x}} \quad (10.7)$$

It is a widely used non-linear activation function which produces an S-shaped curve and the output values are in the range of 0 and 1. It has a vanishing gradient problem, i.e., no change in the prediction for very low input values and very high input values.

5. Bipolar Sigmoid Function

$$\sigma(x) = \frac{1 - e^{-x}}{1 + e^{-x}} \quad (10.8)$$

It outputs values between -1 and +1.

6. Ramp Functions

$$f(x) = \begin{cases} 1 & \text{if } x > 1 \\ x & \text{if } 0 \leq x \leq 1 \\ 0 & \text{if } x < 0 \end{cases} \quad (10.9)$$

It is a linear function whose upper and lower limits are fixed.

7. Tanh – Hyperbolic Tangent Function

The Tanh function is a scaled version of the sigmoid function which is also non-linear. It also suffers from the vanishing gradient problem. The output values range between -1 and 1.

$$\tanh(x) = \frac{2}{1 + e^{-2x}} - 1 \quad (10.10)$$

8. ReLu – Rectified Linear Unit Function

This activation function is a typical function generally used in deep learning neural network models in the hidden layers. It avoids or reduces the vanishing gradient problem. This function outputs a value of 0 for negative input values and works like a linear function if the input values are positive.

$$r(x) = \max(0, x) = \begin{cases} x & \text{if } x \geq 0 \\ 0 & \text{if } x < 0 \end{cases} \quad (10.11)$$

9. Softmax Function

This is a non-linear function used in the output layer that can handle multiple classes. It calculates the probability of each target class which ranges between 0 and 1. The probability of the input belonging to a particular class is computed by dividing the exponential of the given input value by the sum of the exponential values of all the inputs.

$$s(x_i) = \frac{e^{x_i}}{\sum_{j=0}^k e^{x_j}} \text{ where } i = 0 \dots k \quad (10.12)$$

10.4 PERCEPTRON AND LEARNING THEORY

The first neural network model 'Perceptron', designed by Frank Rosenblatt in 1958, is a linear binary classifier used for supervised learning. He modified the McCulloch & Pitts Neuron model by combining two concepts, McCulloch-Pitts model of an artificial neuron and Hebbian learning rule of adjusting weights. He introduced variable weight values and an extra input that represents *bias* to this model. He proposed that artificial neurons could actually learn weights and thresholds from data and came up with a supervised learning algorithm that enabled the artificial neurons to learn the correct weights from training data by itself. The perceptron model (shown in Figure 10.5) consists of 4 steps:

1. Inputs from other neurons
2. Weights and bias
3. Net sum
4. Activation function

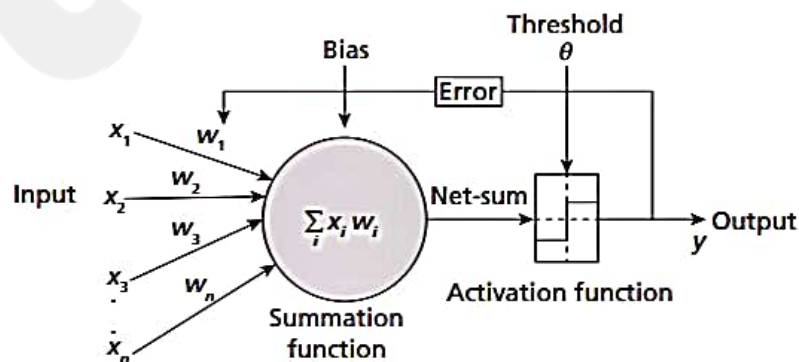


Figure 10.5: Perceptron Model

Thus, the modified neuron model receives a set of inputs $x_1, x_2, \dots, x_n$, their associated weights $w_1, w_2, \dots, w_n$ and a bias. The summation function 'Net-sum' Eq. (10.13) computes the weighted sum of the inputs received by the neuron.

$$\text{Net-sum} = \sum_{i=1}^n x_i w_i \quad (10.13)$$

After computing the 'Net-sum', bias value is added to it and inserted in the activation function as shown below:

$$f(x) = \text{Activation function}(\text{Net-sum} + \text{bias}) \quad (10.14)$$

The activation function is a binary step function which outputs a value 1 if $f(x)$ is above the threshold value θ , and a 0 if $f(x)$ is below the threshold value θ . Then, output of a neuron:

$$Y = \begin{cases} 1 & \text{if } f(x) \geq \theta \\ 0 & \text{if } f(x) < \theta \end{cases} \quad (10.15)$$

Before learning how a neural network works, let us learn about how a perceptron model works.

Algorithm 10.1: Perceptron Algorithm

Set initial weights $w_1, w_2, \dots, w_n$ and bias θ to a random value in the range $[-0.5, 0.5]$.

For each Epoch,

1. Compute the weighted sum by multiplying the inputs with the weights and add the products.
2. Apply the activation function on the weighted sum:

$$Y = \text{Step}((x_1 w_1 + x_2 w_2) - \theta)$$

3. If the sum is above the threshold value, output the value as positive else output the value as negative.
4. Calculate the error by subtracting the estimated output $Y_{\text{estimated}}$ from the desired output Y_{desired} :

$$\text{error } e(t) = Y_{\text{desired}} - Y_{\text{estimated}}$$

[If error $e(t)$ is positive, increase the perceptron output Y and if it is negative, decrease the perceptron output Y .]

5. Update the weights if there is an error:

$$\Delta w_i = \alpha \times e(t) \times x_i$$

$$w_i = w_i + \Delta w_i$$

where, x_i is the input value, $e(t)$ is the error at step t , α is the learning rate and Δw_i is the difference in weight that has to be added to w_i .

Example 10.1: Consider a perceptron to represent the Boolean function AND with the initial weights $w_1 = 0.3$, $w_2 = -0.2$, learning rate $\alpha = 0.2$ and bias $\theta = 0.4$ as shown in Figure 10.6. The activation function used here is the Step function $f(x)$ which gives the output value as binary, i.e., 0 or 1. If value of $f(x)$ is greater than or equal to 0, it outputs 1 or else it outputs 0. Design a perceptron that performs the Boolean function AND and update the weights until the Boolean function gives the desired output.

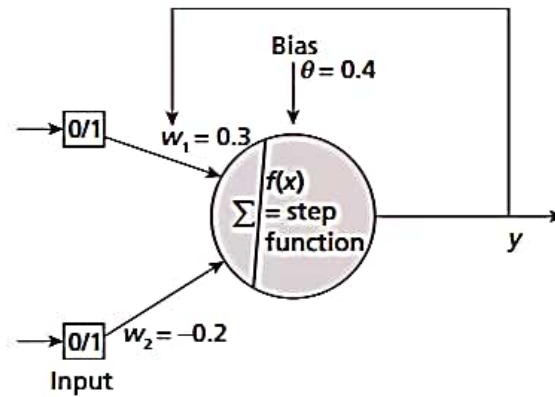


Figure 10.6: Perceptron for Boolean Function AND

Solution: Desired output for Boolean function AND is shown in Table 10.1.

Table 10.1: AND Truth Table

x_1	x_2	Y_{des}
0	0	0
0	1	0
1	0	0
1	1	1

For each Epoch, weighted sum is calculated and the activation function is applied to compute the estimated output Y_{est} . Then, Y_{est} is compared with Y_{des} to find the error. If there is an error, the weights are updated.

Tables 10.2 to 10.5 show how the weights are updated in the four Epochs.

Table 10.2: Epoch 1

Epoch	x_1	x_2	Y_{des}	Y_{est}	Error	w_1	w_2	Status
1	0	0	0	Step $((0 \times 0.3 + 0 \times -0.2) - 0.4) = 0$	0	0.3	-0.2	No change
	0	1	0	Step $((0 \times 0.3 + 1 \times -0.2) - 0.4) = 0$	0	0.3	-0.2	No change
	1	0	0	Step $((1 \times 0.3 + 0 \times -0.2) - 0.4) = 0$	0	0.3	-0.2	No change
	1	1	1	Step $((1 \times 0.3 + 1 \times -0.2) - 0.4) = 0$	1	0.5	0	Change

For input (1, 1) the weights are updated as follows:

$$\Delta w_1 = \alpha \times e(t) \times x_1 = 0.2 \times 1 \times 1 = 0.2$$

$$w_1 = w_1 + \Delta w_1 = 0.3 + 0.2 = 0.5$$

$$\Delta w_2 = \alpha \times e(t) \times x_2 = 0.2 \times 1 \times 1 = 0.2$$

$$w_2 = w_2 + \Delta w_2 = -0.2 + 0.2 = 0$$

Table 10.3: Epoch 2

Epoch	x_1	x_2	Y_{des}	Y_{est}	Error	w_1	w_2	Status
2	0	0	0	Step $((0 \times 0.5 + 0 \times 0) - 0.4) = 0$	0	0.5	0	No change
	0	1	0	Step $((0 \times 0.5 + 1 \times 0) - 0.4) = 0$	0	0.5	0	No change
	1	0	0	Step $((1 \times 0.5 + 0 \times 0) - 0.4) = 1$	-1	0.3	0	Change
	1	1	1	Step $((1 \times 0.3 + 1 \times 0) - 0.4) = 0$	1	0.5	0.2	Change

For input (1, 0) the weights are updated as follows:

$$\Delta w_1 = \alpha \times e(t) \times x_1 = 0.2 \times -1 \times 1 = -0.2$$

$$w_1 = w_1 + \Delta w_1 = 0.5 + \Delta w_1 = 0.5 - 0.2 = 0.3$$

$$\Delta w_2 = \alpha \times e(t) \times x_2 = 0.2 \times -1 \times 0 = 0$$

$$w_2 = w_2 + \Delta w_2 = 0 + \Delta w_2 = 0 + 0 = 0$$

For input (1, 1), the weights are updated as follows:

$$\Delta w_1 = \alpha \times e(t) \times x_1 = 0.2 \times 1 \times 1 = 0.2$$

$$w_1 = w_1 + \Delta w_1 = 0.3 + \Delta w_1 = 0.3 + 0.2 = 0.5$$

$$\Delta w_2 = \alpha \times e(t) \times x_2 = 0.2 \times 1 \times 1 = 0.2$$

$$w_2 = w_2 + \Delta w_2 = 0 + \Delta w_2 = 0 + 0.2 = 0.2$$

Table 10.4: Epoch 3

Epoch	x_1	x_2	Y_{des}	Y_{est}	Error	w_1	w_2	Status
3	0	0	0	Step $((0 \times 0.5 + 0 \times 0.2) - 0.4) = 0$	0	0.5	0.2	No change
	0	1	0	Step $((0 \times 0.5 + 1 \times 0.2) - 0.4) = 0$	0	0.5	0.2	No change
	1	0	0	Step $((1 \times 0.5 + 0 \times 0.2) - 0.4) = 1$	-1	0.3	0.2	Change
	1	1	1	Step $((1 \times 0.3 + 1 \times 0.2) - 0.4) = 1$	0	0.3	0.2	No change

For input (1, 0) the weights are updated as follows:

$$\Delta w_1 = \alpha \times e(t) \times x_1 = 0.2 \times -1 \times 1 = -0.2$$

$$w_1 = w_1 + \Delta w_1 = 0.5 + \Delta w_1 = 0.5 - 0.2 = 0.3$$

$$\Delta w_2 = \alpha \times e(t) \times x_2 = 0.2 \times -1 \times 0 = 0$$

$$w_2 = w_2 + \Delta w_2 = 0 + \Delta w_2 = 0.2 + 0 = 0.2$$

Table 10.5: Epoch 4

Epoch	x_1	x_2	Y_{des}	Y_{est}	Error	w_1	w_2	Status
4	0	0	0	Step $((0 \times 0.3 + 0 \times 0.2) - 0.4) = 0$	0	0.3	0.2	No change
	0	1	0	Step $((0 \times 0.3 + 1 \times 0.2) - 0.4) = 0$	0	0.3	0.2	No change
	1	0	0	Step $((1 \times 0.3 + 0 \times 0.2) - 0.4) = 0$	0	0.3	0.2	No change
	1	1	1	Step $((1 \times 0.3 + 1 \times 0.2) - 0.4) = 1$	0	0.3	0.2	No change

It is observed that with 4 Epochs, the perceptron learns and the weights are updated to 0.3 and 0.2 with which the perceptron gives the desired output of a Boolean AND function.

10.4.1 XOR Problem

A perceptron model can solve all Boolean functions which are linearly separable. However, the XOR problem was identified in 1969 by Minsky and Papert. An XOR function returns a 1, if the two inputs are not equal and a 0 if they are equal. Following is the truth table of an XOR function shown in Table 10.6.

Table 10.6: XOR Truth Table

x_1	x_2	Y
0	0	1
0	1	0
1	0	0
1	1	1

Since XOR is not linearly separable, the single layer perceptron failed to classify and hence this problem led to the evolution of a multi-layer perceptron. Initially, the Multi-Layer Perceptron (MLP) was not a success due to the lack of an appropriate learning algorithm. In 1974, Werbos introduced a back propagation algorithm for the three-layered perceptron network and in 1986, a general back propagation algorithm for a multi-layered perceptron was introduced by Rumelhart and McClelland. Then, again ANN and Deep Neural Networks became a success, solving many complex problems in the current era. Thus, MLP that emerged could solve any non-linear separable problem which is explained later in this chapter.

10.4.2 Delta Learning Rule and Gradient Descent

Generally, learning in neural networks is performed by adjusting the network weights in order to minimize the difference between the desired and estimated outputs. This delta difference is measured as an error function or also called as cost function. The cost function, being linear and continuous, is differentiable. This way of learning called as delta rule (also known as Widrow-Hoff rule or Adaline rule) is a type of back propagation applied for training the network. The training error of a hypothesis is half the squared difference between the desired target output and actual output and is given as follows:

$$\text{Training Error} = \frac{1}{2} \sum_{d \in T} (O_{\text{Desired}} - O_{\text{Estimated}})^2 \quad (10.16)$$

where, T is the training dataset, O_{Desired} and $O_{\text{Estimated}}$ are the desired target output and estimated actual output, respectively, for a training instance d .

The principle of gradient descent is an optimization approach which is used to minimize the cost function by converging to a local minimal point moving in the negative direction of the gradient and each step size during movement is determined by the learning rate and the slope of the gradient.

Gradient descent learning is the foundation of back propagation algorithm used in MLP. Before we study about an MLP, let us first understand the different types of neural networks that differ in their structure, activation function and learning mechanism.

10.5 TYPES OF ARTIFICIAL NEURAL NETWORKS

ANNs consist of multiple neurons arranged in layers. There are different types of ANNs that differ by the network structure, activation function involved and the learning rules used. In an ANN, there are three layers called input layer, hidden layer and output layer. Any general ANN would consist of one input layer, one output layer and zero or more hidden layers.

10.5.1 Feed Forward Neural Network

This is the simplest neural network that consists of neurons which are arranged in layers and the information is propagated only in the forward direction. This model may or may not contain a hidden layer and there is no back propagation. Based on the number of hidden layers they are further classified into single-layered and multi-layered feed forward networks. These ANNs are simple to design and easy to maintain. They are fast but cannot be used for complex learning. They are used for simple classification and simple image processing, etc. The model of a Feed Forward Neural Network is shown in Figure 10.7.

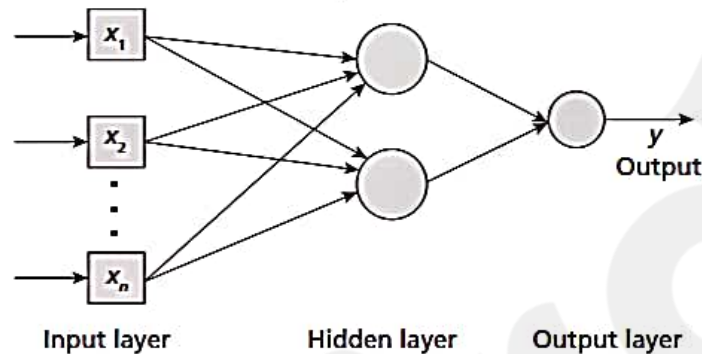


Figure 10.7: Model of a Feed Forward Neural Network

10.5.2 Fully Connected Neural Network

Fully connected neural networks are the ones in which all the neurons in a layer are connected to all other neurons in the next layer. The model of a fully connected neural network is shown in Figure 10.8.

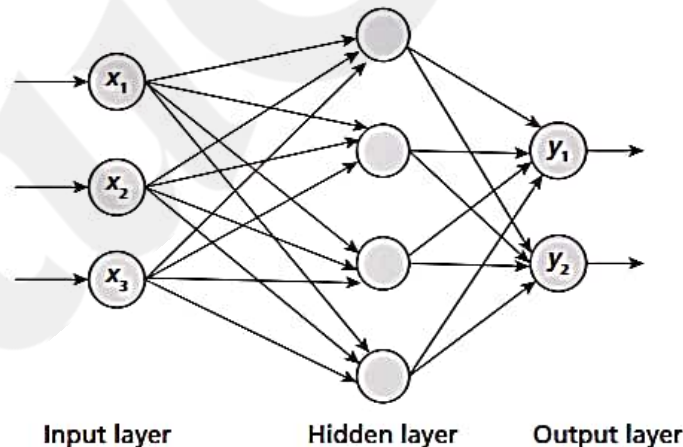


Figure 10.8: Model of a Fully Connected Neural Network

10.5.3 Multi-Layer Perceptron (MLP)

This ANN consists of multiple layers with one input layer, one output layer and one or more hidden layers. Every neuron in a layer is connected to all neurons in the next layer and thus they are fully connected. The information flows in both the directions. In the forward direction, the inputs are multiplied by weights of neurons and forwarded to the activation function of the

neuron and output is passed to the next layer. If the output is incorrect, then in the backward direction, error is back propagated to adjust the weights and biases to get correct output. Thus, the network learns with the training data. This type of ANN is used in deep learning for complex classification, speech recognition, medical diagnosis, forecasting, etc. They are comparatively complex and slow. The model of an MLP is shown in Figure 10.9.

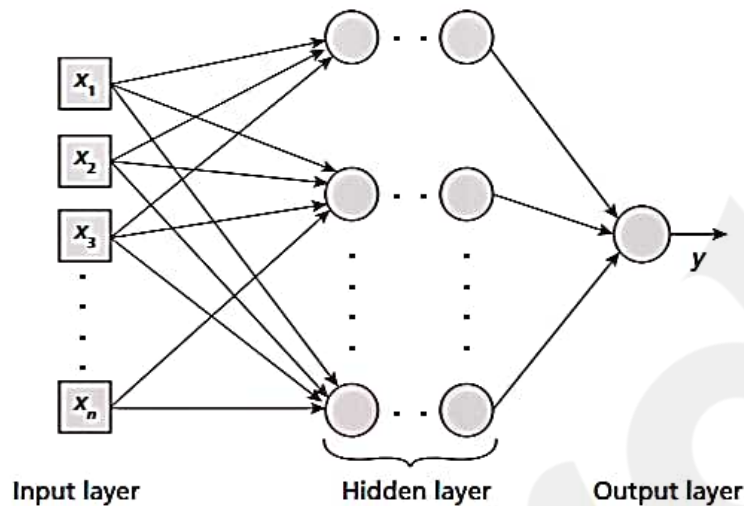


Figure 10.9: Model of a Multi-Layer Perceptron

10.5.4 Feedback Neural Network

Feedback neural networks have feedback connections between neurons that allow information flow in both directions in the network. The output signals can be sent back to the neurons in the same layer or to the neurons in the preceding layers. Hence, this network is more dynamic during training. The model of a feedback neural network is shown in Figure 10.10.

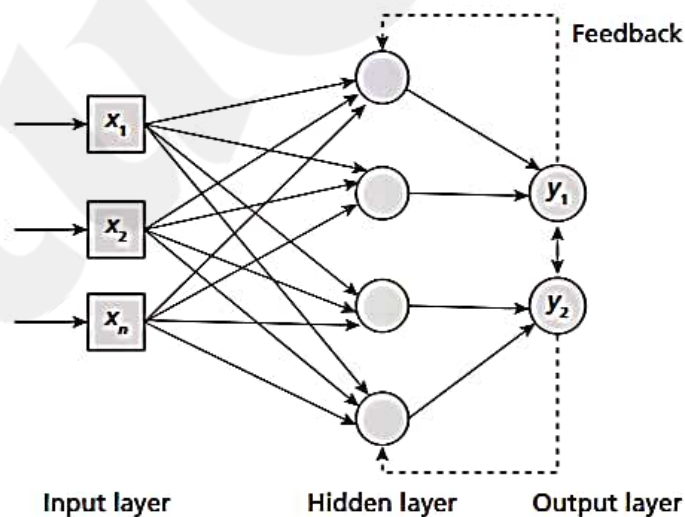


Figure 10.10: Model of a Feedback Neural Network

10.9 POPULAR APPLICATIONS OF ARTIFICIAL NEURAL NETWORKS

ANN learning mechanisms are used in many complex applications that involve modelling of non-linear processes. ANN is a useful model that can handle even noisy and incomplete data. They are used to model complex patterns, recognize patterns and solve prediction problems like humans in many areas such as:

1. Real-time applications: Face recognition, emotion detection, self-driving cars, navigation systems, routing systems, target tracking, vehicle scheduling, etc.
2. Business applications: Stock trading, sales forecasting, customer behaviour modelling, Market research and analysis, etc.
3. Banking and Finance: Credit and loan forecasting, fraud and risk evaluation, currency price prediction, real-estate appraisal, etc.
4. Education: Adaptive learning software, student performance modelling, etc.
5. Healthcare: Medical diagnosis or mapping symptoms to a medical case, image interpretation and pattern recognition, drug discovery, etc.
6. Other Engineering Applications: Robotics, aerospace, electronics, manufacturing, communications, chemical analysis, food research, etc.

10.10 ADVANTAGES AND DISADVANTAGES OF ANN

Advantages of ANN

1. ANN can solve complex problems involving non-linear processes.
2. ANNs can learn and recognize complex patterns and solve problems as humans solve a problem.
3. ANNs have a parallel processing capability and can predict in less time.
4. They have an ability to work with inadequate knowledge. It can even handle incomplete and noisy data.
5. They can scale well to larger data sets and outperforms other learning mechanisms.

Limitations of ANN

1. An ANN requires processors with parallel processing capability to train the network running for many epochs. The function of each node requires a CPU capability which is difficult for very large networks with a large amount of data.
2. They work like a 'black box' and it is exceedingly difficult to understand their working in inner layers. Moreover, it is hard to understand the relationship between the representations learned at each layer.

3. The modelling with ANN is also extremely complicated and the development takes a much longer time.
4. Generally, neural networks require more data than traditional machine learning algorithms, and they do not perform well on small datasets.
5. They are also more computationally expensive than traditional learning techniques.

10.11 CHALLENGES OF ARTIFICIAL NEURAL NETWORKS

The major challenges while modelling a real-time application with ANNs are:

1. Training a neural network is the most challenging part of using this technique. Overfitting or underfitting issues may arise if datasets used for training are not correct. It is also hard to generalize to the real-world data when trained with some simulated data. Moreover, neural network models normally need a lot of training data to be robust and are usable for a real-time application.
2. Finding the weight and bias parameters for neural networks is also hard and it is difficult to calculate an optimal model.

Clustering Algorithms

Cluster analysis is a technique of partitioning a collection of unlabelled objects, with many attributes, into meaningful disjoint groups or clusters. This chapter aims to provide the basic concepts of clustering algorithms.

Learning Objectives

- Introduce the concepts of clustering
- Highlight the role of distance measures in clustering process
- Provide a taxonomy of clustering algorithms
- Explain hierarchical clustering algorithms
- Explain partitional clustering algorithms
- Briefly explain density-based, grid-based, and probabilistic model-based clustering techniques
- Discuss the validation techniques for clustering algorithms

13.1 INTRODUCTION TO CLUSTERING APPROACHES

Cluster analysis is the fundamental task of unsupervised learning. Unsupervised learning involves exploring the given dataset. Cluster analysis is a technique of partitioning a collection of unlabelled objects that have many attributes into meaningful disjoint groups or clusters. This is done using a trial and error approach as there are no supervisors available as in classification. The characteristic of clustering is that the objects in the clusters or groups are similar to each other within the clusters while differ from the objects in other clusters significantly.

The input for cluster analysis is examples or samples. These are known as objects, data points or data instances. All these terms are same and used interchangeably in this chapter. All the samples or objects with no labels associated with them are called unlabelled. The output is

the set of clusters (or groups) of similar data if it exists in the input. For example, the following Figure 13.1(a) shows data points or samples with two features shown in different shaded samples and Figure 13.1(b) shows the manually drawn ellipse to indicate the clusters formed.

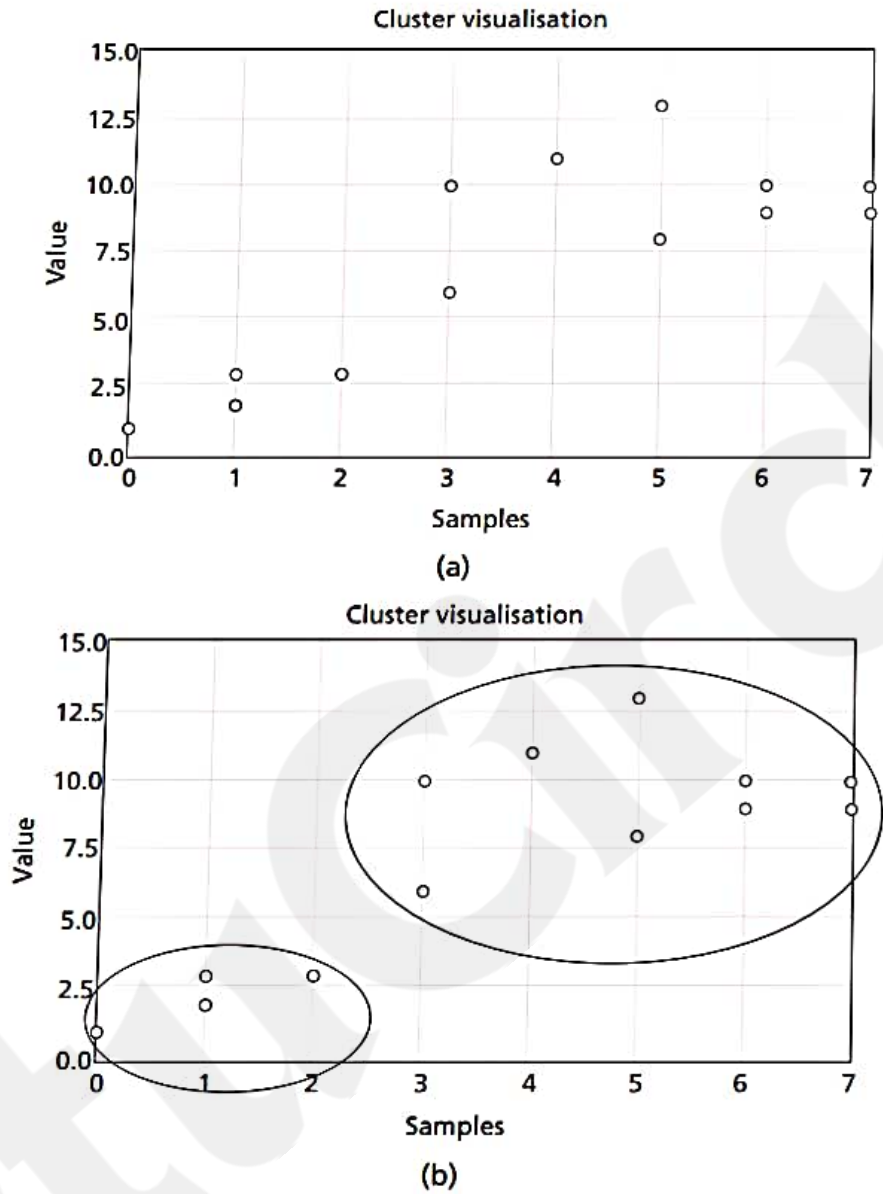


Figure 13.1: (a) Data Samples (b) Clusters' Description

Visual identification of clusters in this case is easy as the examples have only two features. But, when examples have more features, say 100, then clustering cannot be done manually and automatic clustering algorithms are required. Also, automating the clustering process is desirable as these tasks are considered difficult by humans and almost impossible. All clusters are represented by centroids. For example, if the input examples or data is (3, 3), (2, 6) and (7, 9), then the centroid is given as $\left(\frac{3 + 2 + 7}{3}, \frac{3 + 6 + 9}{3}\right) = (4, 6)$. The clusters should not overlap and every cluster should represent only one class. Therefore, clustering algorithms use trial and error method to form clusters that can be converted to labels. Thus, the important differences between classification and clustering are given in Table 13.1.

Table 13.1: Differences between Classification and Clustering

S.No.	Clustering	Classification
1.	Unsupervised learning and cluster formation are done by trial and error as there is no supervisor	Supervised learning with the presence of a supervisor to provide training and testing data
2.	Unlabelled data	Labelled data
3.	No prior knowledge in clustering	Knowledge of the domain is must to label the samples of the dataset
4.	Cluster results are dynamic	Once a label is assigned, it does not change

Applications of Clustering

1. Grouping based on customer buying patterns
2. Profiling of customers based on lifestyle
3. In information retrieval applications (like retrieval of a document from a collection of documents)
4. Identifying the groups of genes that influence a disease
5. Identification of organs that are similar in physiology functions
6. Taxonomy of animals, plants in Biology
7. Clustering based on purchasing behaviour and demography
8. Document indexing
9. Data compression by grouping similar objects and finding duplicate objects

Challenges of Clustering Algorithms

A huge collection of data with higher dimensions (i.e., features or attributes) can pose a problem for clustering algorithms. With the arrival of the internet, billions of data are available for clustering algorithms. This is a difficult task, as scaling is always an issue with clustering algorithms. Scaling is an issue where some algorithms work with lower dimension data but do not perform well for higher dimension data. Also, units of data can pose a problem, like some weights in kg and some in pounds can pose a problem in clustering. Designing a proximity measure is also a big challenge.

The advantages and disadvantages of the cluster analysis algorithms are given in Table 13.2.

Table 13.2: Advantages and Disadvantages of Clustering Algorithms

S.No.	Advantages	Disadvantages
1.	Cluster analysis algorithms can handle missing data and outliers.	Cluster analysis algorithms are sensitive to initialization and order of the input data.
2.	Can help classifiers in labelling the unlabelled data. Semi-supervised algorithms use cluster analysis algorithms to label the unlabelled data and then use classifiers to classify them.	Often, the number of clusters present in the data have to be specified by the user.

(Continued)

S.No.	Advantages	Disadvantages
3.	It is easy to explain the cluster analysis algorithms and to implement them.	Scaling is a problem.
4.	Clustering is the oldest technique in statistics and it is easy to explain. It is also relatively easy to implement.	Designing a proximity measure for the given data is an issue.

13.2 PROXIMITY MEASURES

Scan for 'Additional Information on Proximity Measures'



Clustering algorithms need a measure to find the similarity or dissimilarity among the objects to group them. Similarity and dissimilarity are collectively known as proximity measures. Often, the distance measures are used to find similarity between two objects, say i and j .

Distance measures are known as dissimilarity measures, as these indicate how one object is different from another. Measures like cosine similarity indicate the similarity among objects. Distance measures and similarity measures are two sides of the same coin, as more distance indicates more similarity and vice versa. Distance between two objects, say i and j , is denoted by the symbol D_{ij} .

The properties of the distance measures are:

1. D_{ij} is always positive or zero.
2. $D_{ij} = 0$, i.e., the distance between the object to itself is 0.
3. $D_{ij} = D_{ji}$. This property is called symmetry.
4. $D_{ij} \leq D_{ik} + D_{kj}$. This property is called triangular inequality.

If all these conditions are satisfied, then the distance measure is called a metric.

Based on the data types of the attributes of an object, distance measures vary. The concept of data types is discussed in Chapter 2. It can be recollected that the data types are divided into categorical and quantitative variables. Quantitative variables are numbers and are of two types – nominal and ordinal. For example, gender is a nominal variable as gender can be enumerated as Gender = {male, female}. Ordinal variables look like nominal variables but have an inherent order present in the enumeration. For example, temperature is a nominal variable as temperature can be enumerated as Temperature = {low, medium, high}. One can observe the inherent order present, that is, medium > low and low < medium. Quantitative variables are real or Integer numbers or binary data. In binary data, the attributes of the object can take a Boolean value. Objects whose attributes take binary data are called binary objects.

Let us review some of the proximity measures.

Quantitative Variables

Some of the qualitative variables are discussed below.

Euclidean Distance It is one of the most important and common distance measures. It is also called as L_2 norm. It can be defined as the square root of squared differences between the coordinates of a pair of objects.

The Euclidean distance between objects x_i and x_j with k features is given as follows:

$$\text{Distance}(x_i, x_j) = \sqrt{\sum_{k=1}^n (x_{ik} - x_{jk})^2} \quad (13.1)$$

The advantage of Euclidean distance is that the distance does not change with the addition of new objects. But the disadvantage is that if the units change, the resulting Euclidean or squared Euclidean changes drastically. Another disadvantage is that as the Euclidean distance involves a square root and a square, the computational complexity is high for implementing the distance for millions or billions of operations involved.

City Block Distance City block distance is known as Manhattan distance. This is also known as boxcar, absolute value distance, Manhattan distance, Taxicab or L_1 norm. The formula for finding the distance is given as follows:

$$\text{Distance}(x_i, x_j) = \sum_{k=1}^n |x_{ik} - x_{jk}| \quad (13.2)$$

Chebyshev Distance Chebyshev distance is known as maximum value distance. This is the absolute magnitude of the differences between the coordinates of a pair of objects. This distance is called supremum distance or $L_{\max}$ or L_{∞} norm. The formula for computing Chebyshev distance is given as follows:

$$\text{Distance}(x_i, x_j) = \max_k |x_{ik} - x_{jk}| \quad (13.3)$$

Example 13.1: Suppose, if the coordinates of the objects are (0, 3) and (5, 8), then what is the Chebyshev distance?

Solution: The Euclidean distance using Eq. (13.1) is given as follows:

$$\begin{aligned} \text{Distance}(x_i, x_j) &= \sqrt{(0 - 5)^2 + (3 - 8)^2} \\ &= \sqrt{50} = 7.07 \end{aligned}$$

The Manhattan distance using Eq. (13.2) is given as follows:

$$\text{Distance}(x_i, x_j) = |(0 - 5) + (3 - 8)| = 10$$

The Chebyshev distance using Eq. (13.3) is given as follows:

$$\text{Max} \{|0 - 5|, |3 - 8|\} = \text{Max} \{5, 5\} = 5$$

Minkowski Distance In general, all the above distance measures can be generalized as:

$$\text{Distance}(x_i, x_j) = \left(\sum |x_{ik} - x_{jk}|^r \right)^{\frac{1}{r}} \quad (13.4)$$

This is called Minkowski distance. Here, r is a parameter. When the value of r is 1, the distance measure is called city block distance. When the value of r is 2, the distance measure is called Euclidean distance. When, r is ∞ , then this is Chebyshev distance.

Binary Attributes

Binary attributes have only two values. Distance measures discussed above cannot be applied to find distance between objects that have binary attributes. For finding the distance among objects with binary objects, the contingency Table 13.3 can be used. Let x and y be the objects consisting of N -binary objects. Then, the contingency table can be constructed by counting the number of matching of transitions, 0-0, 0-1, 1-0 and 1-1.

Table 13.3: Contingency Table

Attributes Matching	0	1
0	a	b
1	c	d

In other words, ' a ' is the number of attributes where x attribute is 0 and y attribute is 0. ' b ' is the number of attributes where x attribute is 0 and y attribute is 1, ' c ' is the number of attributes where x attribute is 1 and y attribute is 0 and ' d ' is the number of attributes where x attribute is 1 and y attribute is 1.

Simple Matching Coefficient (SMC) SMC is a simple distance measure and is defined as the ratio of number of matching attributes and the number of attributes. The formula is given as:

$$\frac{a + d}{a + b + c + d} \quad (13.5)$$

The values of a , b , c , and d can be observed from the Table 13.4.

Jaccard Coefficient Jaccard coefficient is another useful measure for and is given as follows:

$$J = \frac{d}{b + c + d} \quad (13.6)$$

Example 13.2: If the given vectors are $x = (1, 0, 0)$ and $y = (1, 1, 1)$ then find the SMC and Jaccard coefficient?

Solution: It can be seen from Table 13.2 that, $a = 0$, $b = 2$, $c = 0$ and $d = 1$.

The SMC using Eq. (13.5) is given as $\frac{a + d}{a + b + c + d} = 0 + 1/3 = 0.33$

Jaccard coefficient using Eq. (13.6) is given as $J = \frac{d}{b + c + d} = 1/3 = 0.33$

Hamming Distance Hamming distance is another useful measure that can be used for knowing the sequence of characters or binary values. It indicates the number of positions at which the characters or binary bits are different.

For example, the hamming distance between $x = (1\ 0\ 1)$ and $y = (1\ 1\ 0)$ is 2 as x and y differ in two positions. The distance between two words, say wood and hood is 1, as they differ in only one character. Sometimes, more complex distance measures like edit distance can also be used.

Categorical Variables

In many cases, categorical values are used. It is just a code or symbol to represent the values. For example, for the attribute Gender, a code 1 can be given to female and 0 can be given to male.

To calculate the distance between two objects represented by variables, we need to find only whether they are equal or not. This is given as:

$$\text{Distance}(x, y) = \begin{cases} 1 & \text{if } x \neq y \\ 0 & \text{if } x = y \end{cases} \quad (13.7)$$

Ordinal Variables

Ordinal variables are like categorical values but with an inherent order. For example, designation is an ordinal variable. If job designation is 1 or 2 or 3, it means code 1 is higher than 2 and code 2 is higher than 3. It is ranked as $1 > 2 > 3$.

Let us assume the designations of office employees are clerk, supervisor, manager and general manager. These can be designated as numbers as clerk = 1, supervisor = 2, manager = 3 and general manager = 4. Then, the distance between employee X who is a clerk and Y who is a manager can be obtained as:

$$\text{Distance}(X, Y) = \frac{|\text{position}(X) - \text{position}(Y)|}{n - 1} \quad (13.8)$$

Here, position(X) and position(Y) indicate the designated numerical value. Thus, the distance between X (Clerk = 1) and Y (Manager = 3) using Eq. (13.8) is given as:

$$\text{Distance}(X, Y) = \frac{|\text{position}(X) - \text{position}(Y)|}{n - 1} = \frac{|1 - 3|}{4 - 1} = \frac{2}{3} \approx 0.66$$

Vector Type Distance Measures

For text classification, vectors are normally used. Cosine similarity is a metric used to measure how similar the documents are irrespective of their size. Cosine similarity measures the cosine of the angle between two vectors projected in a multi-dimensional space. The similarity function for vector objects can be defined as:

$$\text{sim}(X, Y) = \frac{X \cdot Y}{\|X\| \times \|Y\|} = \frac{\sum_{i=1}^n X_i \times Y_i}{\sqrt{\sum_{i=1}^n X_i^2} \times \sqrt{\sum_{i=1}^n Y_i^2}} \quad (13.9)$$

The numeration is the dot product of the vectors A and B. The denominator is the product of the norm of vectors A and B.

Example 13.3: If the given vectors are $A = \{1, 1, 0\}$ and $B = \{0, 1, 1\}$, then what is the cosine similarity?

Solution: The dot product of the vector is $1 \times 0 + 1 \times 1 + 0 \times 1 = 1$. The norm of the vectors A and B is $\sqrt{2}$.

So, the cosine similarity using Eq. (13.9) is given as $\frac{1}{\sqrt{2}\sqrt{2}} = \frac{1}{2} = 0.5$

Now, let us discuss the types of clustering algorithms, which include hierarchical, partitional, density-based and grid-based algorithms.

Scan for information on 'Taxonomy of Clustering Algorithms'



13.3 HIERARCHICAL CLUSTERING ALGORITHMS

Hierarchical methods produce a nested partition of objects with hierarchical relationships among objects. Often, the hierarchy relationship is shown in the form of a dendrogram.

Hierarchical methods include categories, agglomerative methods and divisive methods. In agglomerative methods, initially all individual samples are considered as a cluster, that is, a cluster with a single element. Then, they are merged and the process is continued to get a single cluster. Divisive methods use another kind of philosophy, where a single cluster of all samples of the dataset taken initially is chosen and then partitioned. This partition process is continued until the cluster is split into smaller clusters.

Agglomerative methods merge clusters to reduce the number of clusters. This is repeated each time while merging two closest clusters to get a single cluster. The procedure of agglomerative clustering is given as follows:

Algorithm 13.1: Agglomerative Clustering

1. Place each N sample or data instance into a separate cluster. So, initially N clusters are available.
2. Repeat the following steps until a single cluster is formed:
 - (a) Determine two most similar clusters.
 - (b) Merge the two clusters into a single cluster reducing the number of clusters as $N-1$.
3. Choose resultant clusters of step 2 as result.

All the clusters that are produced by hierarchical algorithms have equal diameters. The main disadvantage of this approach is that once the cluster is formed, it is an irreversible decision.

13.3.1 Single Linkage or MIN Algorithm

Hierarchical clustering algorithms produce nested clusters, which can be visualized as a hierarchical tree or dendrogram. The idea behind this approach is proximity among clusters. In single linkage algorithm, the smallest distance (x, y) , where x is from one cluster and y is from another cluster, is the distance between all possible pairs of the two groups or clusters (or simply the smallest distance of two points where points are in different clusters) and is used for merging the clusters. This corresponds to finding of minimum spanning tree (MST) of a graph.

The distance measures between individual samples or data points is already demonstrated in the previous section 13.2. To understand the single linkage algorithm, go through the following numerical problem that involves finding the distance between clusters.

Example 13.4: Consider the array of points as shown in the following Table 13.4.

Table 13.4: Sample Data

Objects	X	Y
0	1	4
1	2	8
2	5	10
3	12	18
4	14	28

Apply single linkage algorithm.

Solution: The similarity table among the variables is computed as shown in Table 13.4. Euclidean distance is computed as shown in the following Table 13.5.

Table 13.5: Proximity Matrix

Objects	0	1	2	3	4
0	–	4.123	7.211	17.804	27.295
1		–	3.606	14.142	23.324
2			–	10.630	20.124
3				–	10.198
4					–

The minimum distance is 3.606. Therefore, the items 1 and 2 are clustered together. The resultant is shown in Table 13.6.

Table 13.6: After Iteration 1

Clusters	{1, 2}	0	3	4
{1, 2}	–	4.123	10.630	20.124
0		–	17.804	27.295
3			–	10.198
4				–

The distance between the group {1, 2} and items {0}, {3} and {4} is computed using the formula:

$$D_{SL}(C_i, C_j) = \text{minimum}_{a \in C_i, b \in C_j} d(a, b) \quad (13.10)$$

Here, D_{SL} is the single linkage distance, C_i, C_j are clusters and $d(a, b)$ is the distance between the elements a and b .

Thus, the distance between {1, 2} and {0} is:

$$\text{Minimum } \{d(1, 0), d(2, 0)\} = \text{Minimum } \{4.123, 7.211\} = 4.123$$

The distance between {1, 2} and {3} is given as:

$$\text{Minimum } \{d(1, 3), d(2, 3)\} = \text{Minimum } \{14.142, 10.630\} = 10.630$$

The distance between {1, 2} and {4} is given as:

$$\text{Minimum } \{d(1, 4), d(2, 4)\} = \text{Minimum } \{23.324, 20.124\} = 20.124$$

This is shown in Table 13.6. The minimum distance of Table 13.6 is 4.123.

Therefore, {0, 1, 2} is clustered together. This result is shown in Table 13.7.

Table 13.7: After Iteration 2

Clusters	{0, 1, 2}	3	4
{0, 1, 2}	–	10.630	20.124
3		–	10.198
4			–

Thus, the distance between {0, 1, 2} and {3} using Eq. (13.10) is given as

$$\text{Minimum } \{ \{0, 3\}, \{1, 3\}, \{2, 3\} \} = \text{Minimum } \{ 17.804, 14.142, 10.630 \} = 10.630.$$

Thus, the distance between {0, 1, 2} and {4} is:

$$\text{Minimum } \{ \{0, 4\}, \{1, 4\}, \{2, 4\} \} = \text{Minimum } \{ 27.295, 23.324, 20.124 \} = 20.124.$$

This is shown in Table 13.7. The minimum is 10.198.

Therefore, the items {3, 4} are merged. And, the items {1, 2, 3} and {4, 5} are merged. The resultant is shown in Table 13.8.

Table 13.8: After Iteration 3

Clusters	{0, 1, 2}	{3, 4}
{0, 1, 2}	–	?
{3, 4}		–

The computation of ? in Table 13.8 is not needed as no other items are left. Therefore, the clusters {0, 1, 2} and {3, 4} are merged.

Dendrograms are used to plot the hierarchy of clusters. Dendrogram for the above clustering is shown in the following Figure 13.2.

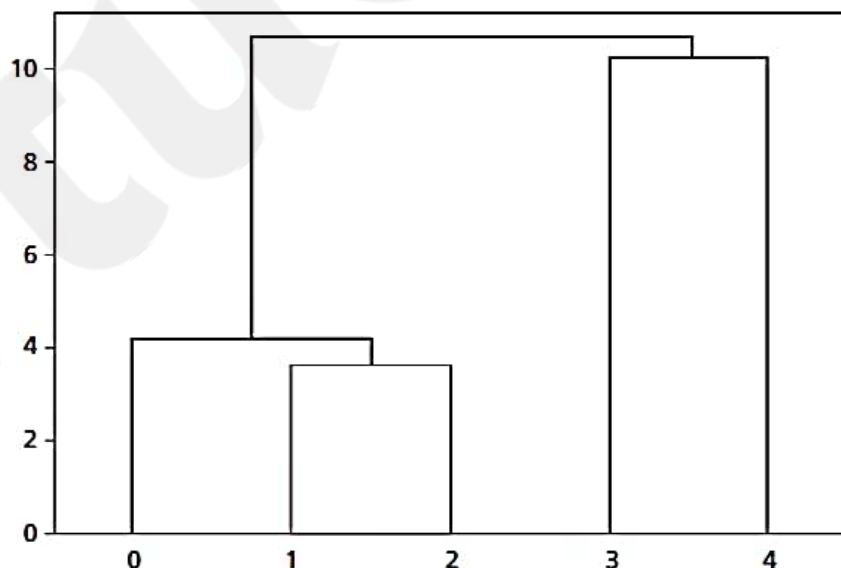


Figure 13.2: Dendrogram for Table 13.4

13.3.2 Complete Linkage or MAX or Clique

In complete linkage algorithm, the distance (x, y) , (where x is from one cluster and y is from another cluster), is the largest distance between all possible pairs of the two groups or clusters (or simply the largest distance of two points where points are in different clusters) as given below. It is used for merging the clusters.

$$D_{CL}(C_i, C_j) = \text{maximum}_{a \in C_i, b \in C_j} d(a, b) \quad (13.11)$$

Dendrogram for the above clustering is shown in Figure 13.3.

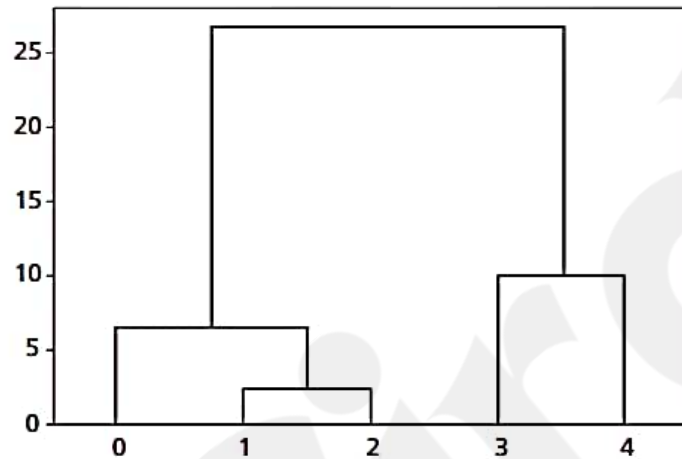


Figure 13.3: Dendrogram for Table 13.4

13.3.3 Average Linkage

In case of an average linkage algorithm, the average distance of all pairs of points across the clusters is used to form clusters. The average value computed between clusters C_i, C_j is given as follows:

$$D_{AL}(C_i, C_j) = \frac{1}{m_i m_j} \sum_{a \in C_i, b \in C_j} d(a, b) \quad (13.12)$$

Here, m_i and m_j are sizes of the clusters.

The dendrogram for Table 13.4 is given in Figure 13.4.

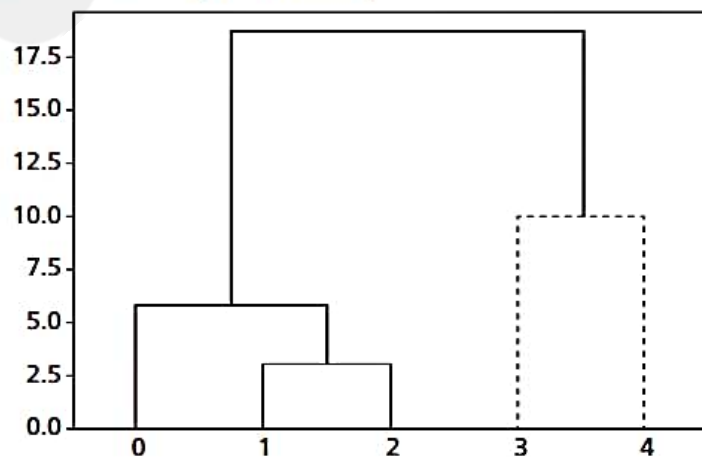


Figure 13.4: Dendrogram for Table 13.4

13.3.4 Mean-Shift Clustering Algorithm

Mean-shift is a non-parametric and hierarchical clustering algorithm. This algorithm is also known as mode seeking algorithm or a sliding window algorithm. It has many applications in image processing and computer vision.

There is no need for any prior knowledge of clusters or shape of the clusters present in the dataset. The algorithm slowly moves from its initial position towards the dense regions.

The algorithm uses a window, which is basically a weighting function. Gaussian window is a good example of a window. The radius of the kernel is called bandwidth. The entire window is called a kernel. The window is based on the concept of kernel density function and its aim is to find the underlying data distribution. The method of calculation of mean is dependent on the choice of windows. If a Gaussian window is chosen, then every point is assigned a weight that decreases as the distance from the kernel center increases. The algorithm is given below.

Algorithm 13.2: Mean-Shift Clustering

Step 1: Design a window.

Step 2: Place the window on a set of data points.

Step 3: Compute the mean for all the points that come under the window.

Step 4: Move the center of the window to the mean computed in step 3. Thus, the window moves towards the dense regions. The movement to the dense region is controlled by a mean shift vector. The mean shift vector is given as:

$$v_s = \frac{1}{K} \sum_{x_i \in S_k} (x_i - x) \quad (13.13)$$

Here, K is the number of points and S_k is the data points where the distance from data points x_i and centroid of the kernel x is within the radius of the sphere. Then, the centroid is updated as $x = x + v_s$.

Step 5: Repeat the steps 3–4 for convergence. Once convergence is achieved, no further points can be accommodated.

Advantages

1. No model assumptions
2. Suitable for all non-convex shapes
3. Only one parameter of the window, that is, bandwidth is required
4. Robust to noise
5. No issues of local minima or premature termination

Disadvantages

1. Selecting the bandwidth is a challenging task. If it is larger, then many clusters are missed. If it is small, then many points are missed and convergence occurs as the problem.
2. The number of clusters cannot be specified and user has no control over this parameter.

Scan for 'Additional Examples'



13.4 PARTITIONAL CLUSTERING ALGORITHM

'k-means' algorithm is a straightforward iterative partitional algorithm. Here, k stands for the user specified requested clusters as users are not aware of the clusters that are present in the dataset. The k -means algorithm assumes that the clusters do not overlap. Therefore, a sample or data point can belong to only one cluster in the end. Also, this algorithm can detect clusters of shapes like circular or spherical.

Initially, the algorithm needs to be initialized. The algorithm can select k data points randomly or use the prior knowledge of the data. In most cases, in k -means algorithm setup, prior knowledge is absent. The composition of the cluster is based on the initial condition, therefore, initialization is an important task. The sample or data points need to be normalized for better performance. The concepts of normalization are covered in Chapter 3.

The core process of the k -mean algorithm is assigning a sample to a cluster, that is, assigning each sample or data point to the k cluster centers based on its distance and the centroid of the clusters. This distance should be minimum. As a new sample is added, new computation of mean vectors of the points for that cluster to which sample is assigned is required. Therefore, this iterative process is continued until no change of instances to clusters is noticed. This algorithm then terminates and the termination is guaranteed.

Algorithm 13.3: k-means Algorithm

Step 1: Determine the number of clusters before the algorithm is started. This is called k .

Step 2: Choose k instances randomly. These are initial cluster centers.

Step 3: Compute the mean of the initial clusters and assign the remaining sample to the closest cluster based on Euclidean distance or any other distance measure between the instances and the centroid of the clusters.

Step 4: Compute new centroid again considering the newly added samples.

Step 5: Perform the steps 3–4 till the algorithm becomes stable with no more changes in assignment of instances and clusters.

k -means can also be viewed as greedy algorithm as it involves partitioning n samples to k clusters to minimize Sum of Squared Error (SSE). SSE is a metric that is a measure of error that gives the sum of the squared Euclidean distances of each data to its closest centroid. It is given as:

$$SSE = \sum_{i=1}^k \text{dist}(c_i, x)^2 \quad (13.14)$$

Here, c_i is the centroid of the i^{th} cluster, x is the sample or data point and dist is the Euclidean distance. The aim of the k -means algorithm is to minimize SSE.

Advantages

1. Simple
2. Easy to implement

Disadvantages

1. It is sensitive to initialization process as change of initial points leads to different clusters.
2. If the samples are large, then the algorithm takes a lot of time.

How to Choose the Value of k ?

It is obvious that k is the user specified value specifying the number of clusters that are present. Obviously, there are no standard rules available to pick the value of k . Normally, the k -means algorithm is run with multiple values of k and within group variance (sum of squares of samples with its centroid) and plotted as a line graph. This plot is called Elbow curve. The optimal or best value of k can be determined from the graph. The optimal value of k is identified by the flat or horizontal part of the Elbow curve.

Complexity

The complexity of k -means algorithm is dependent on the parameters like n , the number of samples, k , the number of clusters, $\Theta(nkId)$. I is the number of iterations and d is the number of attributes. The complexity of k -means algorithm is $O(n^2)$.

Example 13.5: Consider the following set of data given in Table 13.9. Cluster it using k -means algorithm with the initial value of objects 2 and 5 with the coordinate values (4, 6) and (12, 4) as initial seeds.

Table 13.9: Sample Data

Objects	X-coordinate	Y-coordinate
1	2	4
2	4	6
3	6	8
4	10	4
5	12	4

Solution: As per the problem, choose the objects 2 and 5 with the coordinate values. Hereafter, the objects' id is not important. The samples or data points (4, 6) and (12, 4) are started as two clusters as shown in Table 13.10.

Initially, centroid and data points are same as only one sample is involved.

Table 13.10: Initial Cluster Table

Cluster 1	Cluster 2
(4, 6)	(12, 4)
Centroid 1 (4, 6)	Centroid 2 (12, 4)

Iteration 1: Compare all the data points or samples with the centroid and assign to the nearest sample. Take the sample object 1 (2, 4) from Table 13.9 and compare with the centroid of

the clusters in Table 13.10. The distance is 0. Therefore, it remains in the same cluster. Similarly, consider the remaining samples. For the object 1 (2, 4), the Euclidean distance between it and the centroid is given as:

$$\text{Dist (1, centroid 1)} = \sqrt{(2-4)^2 + (4-6)^2} = \sqrt{8}$$

$$\text{Dist (1, centroid 2)} = \sqrt{(2-12)^2 + (4-4)^2} = \sqrt{100} = 10$$

Object 1 is closer to the centroid of cluster 1 and hence assign it to cluster 1. This is shown in Table 13.11. Object 2 is taken as centroid point.

For the object 3 (6, 8), the Euclidean distance between it and the centroid points is given as:

$$\text{Dist (3, centroid 1)} = \sqrt{(6-4)^2 + (8-6)^2} = \sqrt{8}$$

$$\text{Dist (3, centroid 2)} = \sqrt{(6-12)^2 + (8-4)^2} = \sqrt{52}$$

Object 3 is closer to the centroid of cluster 1 and hence remains in the same cluster 1.

Proceed with the next point object 4(10, 4) and again compare it with the centroids in Table 13.10.

$$\text{Dist (4, centroid 1)} = \sqrt{(10-4)^2 + (4-6)^2} = \sqrt{40}$$

$$\text{Dist (4, centroid 2)} = \sqrt{(10-12)^2 + (4-4)^2} = \sqrt{4} = 2$$

Object 4 is closer to the centroid of cluster 2 and hence assign it to the cluster table. Object 4 is in the same cluster. The final cluster table is shown in Table 13.11.

Obviously, Object 5 is in Cluster 3. Recompute the new centroids of cluster 1 and cluster 2. They are (4, 6) and (11, 4), respectively.

Table 13.11: Cluster Table After Iteration 1

Cluster 1	Cluster 2
(4, 6)	(10, 4)
(2, 4)	(12, 4)
(6, 8)	
Centroid 1 (4, 6)	Centroid 2 (11, 4)

The second iteration is started again with the Table 13.11.

Obviously, the point (4, 6) remains in cluster 1, as the distance of it with itself is 0. The remaining objects can be checked. Take the sample object 1 (2, 4) and compare with the centroid of the clusters in Table 13.12.

$$\text{Dist (1, centroid 1)} = \sqrt{(2-4)^2 + (4-6)^2} = \sqrt{8}$$

$$\text{Dist (1, centroid 2)} = \sqrt{(2-11)^2 + (4-4)^2} = \sqrt{81} = 9$$

Object 1 is closer to centroid of cluster 1 and hence remains in the same cluster. Take the sample object 3 (6, 8) and compare with the centroid values of clusters 1 (4, 6) and cluster 2 (11, 4) of the Table 13.12.

$$\text{Dist (3, centroid 1)} = \sqrt{(6-4)^2 + (8-6)^2} = \sqrt{8}$$

$$\text{Dist (3, centroid 2)} = \sqrt{(6-11)^2 + (8-4)^2} = \sqrt{41}$$

Object 3 is closer to centroid of cluster 1 and hence remains in the same cluster. Take the sample object 4 (10, 4) and compare with the centroid values of clusters 1 (4, 6) and cluster 2 (11, 4) of the Table 13.12:

$$\text{Dist (4, centroid 1)} = \sqrt{(10 - 4)^2 + (4 - 6)^2} = \sqrt{40}$$

$$\text{Dist (3, centroid 2)} = \sqrt{(10 - 11)^2 + (4 - 4)^2} = \sqrt{1} = 1$$

Object 3 is closer to centroid of cluster 2 and hence remains in the same cluster. Obviously, the sample (12, 4) is closer to its centroid as shown below:

$$\text{Dist (5, centroid 1)} = \sqrt{(12 - 4)^2 + (4 - 6)^2} = \sqrt{68}$$

$\text{Dist (5, centroid 2)} = \sqrt{(12 - 11)^2 + (4 - 4)^2} = \sqrt{1} = 1$. Therefore, it remains in the same cluster. Object 5 is taken as centroid point.

The final cluster Table 13.12 is given below:

Table 13.12: Cluster Table After Iteration 2

Cluster 1	Cluster 2
(4, 6)	(10, 4)
(2, 4)	(12, 4)
(6, 8)	
Centroid (4, 6)	Centroid (11, 4)

There is no change in the cluster Table 13.12. It is exactly the same; therefore, the *k*-means algorithm terminates with two clusters with data points as shown in the Table 13.12.

Scan for 'Additional Examples'



13.5 DENSITY-BASED METHODS

Density-based spatial clustering of applications with noise (DBSCAN) is one of the density-based algorithms. Density of a region represents the region where many points above the specified threshold are present. In a density-based approach, the clusters are regarded as dense regions of objects that are separated by regions of low density such as noise. This is same as a human's intuitive way of observing clusters.

The concept of density and connectivity is based on the local distance of neighbours. The functioning of this algorithm is based on two parameters, the size of the neighbourhood (ϵ) and the minimum number of points (m).

1. Core point – A point is called a core point if it has more than specified number of points (m) within ϵ -neighbourhood.
2. Border point – A point is called a border point if it has fewer than ' m ' points but is a neighbour of a core point.
3. Noise point – A point that is neither a core point nor border point.

The main idea is that every data point or sample should have atleast a minimum number of neighbours in a neighbourhood. The neighbourhood of radius ϵ should have atleast m points. The notion of density connectedness determines the quality of the algorithm.

The following connectedness measures are used for this algorithm.

1. Direct density reachable – The point X is directly reachable from Y , if:
 - (a) X is the ϵ -neighborhood of Y
 - (b) Y is a core point
2. Densely reachable – The point X is densely reachable from Y , if there is a set of core points that leads from Y to X .
3. Density connected – X and Y are densely connected if Z is a core point and thus points X and Y are densely reachable from Z .

Algorithm 13.4: DBSCAN

Step 1: Randomly select a point p . Compute distance between p and all other points.

Step 2: Find all points from p with respect to its neighbourhood and check whether it has minimum number of points m . If so, it is marked as a core point.

Step 3: If it is a core point, then a new cluster is formed, or existing cluster is enlarged.

Step 4: If it is a border point, then the algorithm moves to the next point and marks it as visited.

Step 5: If it is a noise point, they are removed.

Step 6: Merge the clusters if it is mergeable, $\text{dist}(c_i, c_j) < \epsilon$.

Step 7: Repeat the process 3–6 till all points are processed.

Advantages

1. No need for specifying the number of clusters beforehand
2. The algorithm can detect clusters of any shapes
3. Robust to noise
4. Few parameters are needed

The complexity of this algorithm is $O(n \log n)$.

13.6 GRID-BASED APPROACH

Grid-based approach is a space-based approach. It partitions space into cells, the given data is fitted on the cells for cluster formation.

There are three important concepts that need to be mastered for understanding the grid-based schemes. They are:

1. Subspace clustering
2. Concept of dense cells
3. Monotonicity property

Let us discuss about them.

Subspace Clustering

Grid-based algorithms are useful for clustering high-dimensional data, that is, data with many attributes. Some data like gene data may have millions of attributes. Every attribute is called a dimension. But all the attributes are not needed, as in many applications one may not require all the attributes. For example, an employee's address may not be required for profiling his diseases. Age may be required in that case. So, one can conclude that only a subset of features is required. For example, one may be interested in grouping gene data with similar characteristics or organs that have similar functions.

Finding subspaces is difficult. For example, N dimensions may have 2^{N-1} subspaces. Exploring all the subspaces is a difficult task. Here, only the CLIQUE algorithms are useful for exploring the subspaces. CLIQUE (Clustering in Quest) is a grid-based method for finding clustering in subspaces. CLIQUE uses a multiresolution grid data structure.

Concept of Dense Cells

CLIQUE partitions each dimension into several overlapping intervals and intervals it into cells. Then, the algorithm determines whether the cell is dense or sparse. The cell is considered dense if it exceeds a threshold value, say τ . Density is defined as the ratio of number of points and volume of the region. In one pass, the algorithm finds the number of cells, number of points, etc. and then combines the dense cells. For that, the algorithm uses the contiguous intervals and a set of dense cells.

Algorithm 13.5: Dense Cells

- Step 1:** Define a set of grid points and assign the given data points on the grid.
- Step 2:** Determine the dense and sparse cells. If the number of points in a cell exceeds the threshold value τ , the cell is categorized as dense cell. Sparse cells are removed from the list.
- Step 3:** Merge the dense cells if they are adjacent.
- Step 4:** Form a list of grid cells for every subspace as output.

Monotonicity Property

CLIQUE uses anti-monotonicity property or apriori property of the famous apriori algorithm. It means that all the subsets of a frequent item should be frequent. Similarly, if the subset is infrequent, then all its supersets are infrequent as well. Based on the apriori property, one can conclude that a k -dimensional cell has r points if and only if every $(k - 1)$ dimensional projections of this cell have at least r points. So like association rule mining that uses apriori rule, the candidate dense cells are generated for higher dimensions. The algorithm works in two stages as shown below.

Algorithm 13.6: CLIQUE

Stage 1:

- Step 1:** Identify the dense cells.
- Step 2:** Merge dense cells c_1 and c_2 if they share the same interval.

(Continued)

Step 3: Generate Apriori rule to generate $(k + 1)^{\text{th}}$ cell for higher dimension. Then, check whether the number of points cross the threshold. This is repeated till there are no dense cells or new generation of dense cells.

Stage 2:

Step 1: Merging of dense cells into a cluster is carried out in each subspace using maximal regions to cover dense cells. The maximal region is an hyperrectangle where all cells fall into.

Step 2: Maximal region tries to cover all dense cells to form clusters.

In stage two, CLIQUE starts from dimension 2 and starts merging. This process is continued till the n -dimension.

Advantages of CLIQUE

1. Insensitive to input order of objects
2. No assumptions of underlying data distributions
3. Finds subspace of higher dimensions such that high-density clusters exist in those subspaces

Disadvantage

The disadvantage of CLIQUE is that tuning of grid parameters, such as grid size, and finding optimal threshold for finding whether the cell is dense or not is a challenge.